

CS 3001 – Computer Networks

Chapter 1 Complete Notes & Assignment 1 Solutions

FAST NUCES Lahore — BCS 5D / 5E

Fall 2026

Contents

Lecture 01 — Introduction	2
1. What is the Internet?	2
2. What is a protocol?	3
3. Network Edge: Hosts as Client, Server, or Peer	3
No formulas yet	4
Lecture 02 — Chapter 1 (continued)	5
1. Access Networks & Physical Media	5
2. Switching — how data actually moves through the network core	6
3. Numerical Example — Circuit-Switched File Transfer	8
4. Packet switching vs. circuit switching — capacity comparison	8
Formula Summary — Lecture 02	9
Lecture 03 — Chapter 1 (continued)	10
1. Internet Structure: “A Network of Networks”	10
2. Performance Metrics: Delay, Loss, Throughput	11
3. Throughput	14
4. Bandwidth-Delay Product (BDP)	14
Formula Summary — Lecture 03	15
Lecture 04 — Chapter 1 (continued)	17
1. Network Security	17
2. Protocol Layers and Reference Models	19
3. Additional Material	21
Formula Summary — Lecture 04	22
Assignment #1 — Solutions	23
PART 1 — Textbook Questions (Kurose & Ross, 8th Edition, Chapter 1)	23
PART 2	25

Lecture 01 — Introduction

CS 3001 Computer Networks | Chapter 1 (Kurose & Ross)

Course logistics (office hours, grading %, textbook, TA names) aren't included here since they're not exam content — this covers only the conceptual material, following the slide order.

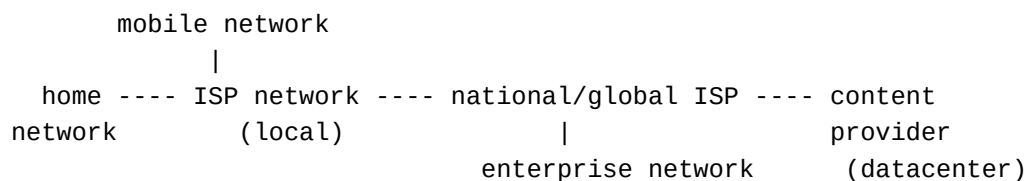
1. What is the Internet?

There are two ways to look at the same thing: the **hardware view** (“nuts and bolts”) and the **service view**.

1.1 The “nuts and bolts” view — what it’s physically made of

Think of the Internet as billions of devices connected together, plus the wires/radio waves that connect them, plus the rules everyone follows. Break it into four pieces:

- **Hosts (end systems):** Any device that runs an application and talks over the network — your laptop, phone, a Netflix server, a smart fridge. “End system” just means it sits at the *edge* of the network, not in the middle forwarding other people’s traffic.
- **Packet switches (routers & switches):** These sit in the middle of the network. Their only job is to grab incoming chunks of data (“packets”) and forward them toward their destination — like a sorting hub in a courier network.
- **Communication links:** The physical medium data travels through — fiber, copper cable, radio waves, satellite. Each link has a **transmission rate (bandwidth)** — how many bits per second it can push through, e.g. 100 Mbps.
- **Networks:** A collection of devices, routers, and links managed by one organization (e.g., your university’s network, your ISP’s network). The Internet is literally a “network of networks” — thousands of individually-run networks all agreeing to talk to each other.



(Simplified: your home/phone connects to a local ISP, which connects to bigger regional/national ISPs, which all connect to each other and to big companies like Google/Netflix that run their own datacenters.)

1.2 The “services” view — what it does for you

Forget the wires for a second — from an app developer’s point of view, the Internet is just **infrastructure that lets applications talk to each other**: web browsing, streaming, email, gaming, e-commerce. It gives programmers a standard “hook” (an API/interface) to plug into so their app can send and receive data, the same way the postal service gives you a mailbox instead of making you build your own delivery trucks.

1.3 A textbook (Wikipedia) definition, in plain words

The Internet is a global system of interconnected networks that all agree to use the same rulebook (the TCP/IP protocol suite) so any two devices anywhere can talk to each other.

2. What is a protocol?

Protocol = an agreed-upon set of rules for communication. Nothing more mysterious than that.

Human analogy: When two people meet, there's an unwritten protocol — 1. Person A says “Hi” 2. Person B replies “Hi” 3. Person A asks “Got the time?” 4. Person B answers “2:00”

If B doesn't respond to “Hi,” the “protocol” breaks and A knows something's wrong (ignored, or B is busy).

Network version of the exact same idea — your browser and a web server go through a similar handshake: 1. Browser → Server: “TCP connection request” (basically saying “Hi, can we talk?”) 2. Server → Browser: “TCP connection response” (“Hi, yes we can”) 3. Browser → Server: GET http://gaia.cs.umass.edu/kurose_ross (this is the “got the time?” — a specific request) 4. Server → Browser: sends back the actual webpage file (this is the “2:00” — the answer)

So a **protocol formally defines:** what messages look like (format), what order they're sent/received in, and what each side should do when a message arrives (or doesn't).

3. Network Edge: Hosts as Client, Server, or Peer

The “network edge” is just a fancy term for **all the end devices** (hosts) sitting at the outer boundary of the network — as opposed to the routers in the middle. Every host acting on the network plays one of three roles:

Role	What it does	Characteristics	Examples
Server	Provides a service / resource to others	Always on, has a permanent (fixed) IP address, usually lives in a big data center. Can serve many roles: web server, mail server, DNS server, print server	Google's search servers, a university's mail server
Client	Requests a service from a server	May be turned on/off intermittently (not always-on), may get a different IP address each time it connects, clients don't talk directly to each other	Your laptop browsing a website
Peer	Both requests AND provides service — acts as client and server at the same time	No dedicated always-on server; any peer can talk directly to any other peer; peers come and go, complex to manage since there's no central authority	Skype (old P2P version), BitTorrent, Napster

Why this distinction matters: In a client-server setup (like Gmail), Google's servers are always there waiting — if they go down, nobody can use Gmail. In a peer-to-peer setup (like BitTorrent), there's no single point that can go down and kill the whole system, because every peer helps serve everyone else, but it's harder to manage/secure since there's no central "boss."

No formulas yet

Lecture 01 is purely conceptual — no numericals or formulas were introduced. The math (transmission delay, propagation delay, throughput, BDP) starts from Lecture 02 onward.

Lecture 02 — Chapter 1 (continued)

CS 3001 Computer Networks

1. Access Networks & Physical Media

The question this section answers: How does your device actually plug into the Internet?

1.1 Types of access networks

- **Residential** (your home Wi-Fi/broadband)
- **Institutional** (your university/office network)
- **Mobile** (Wi-Fi, 4G/5G on your phone)

1.2 Cable-based access (the internet-over-cable-TV wire)

Your cable TV wire is split into many frequency “channels” — some channels carry TV, some carry Internet data. This trick of splitting one wire into multiple channels by frequency is called **FDM (Frequency Division Multiplexing)** — think of it like a single road painted with multiple lanes, each lane (frequency band) carrying a different “kind” of traffic simultaneously.

Your modem → cable splitter → shared cable → CMTS (cable company's box) → ISP → Internet

- Called **HFC (Hybrid Fiber Coax)** — part of the path is fiber, part is old-style coaxial copper cable.
- It's **asymmetric**: downstream (downloading) is much faster (up to 40 Mbps–1.2 Gbps) than upstream (uploading, 30–100 Mbps) — because most home users download way more than they upload.
- **Key catch**: the cable is *shared* among all homes in the neighborhood connected to the same headend — so your speed can drop when your neighbors are all streaming at once.

1.3 DSL (Digital Subscriber Line — internet over your landline phone wire)

Uses your existing telephone copper wire, but splits it so voice calls and internet data travel over different frequencies on the *same* wire simultaneously (so you can be on a call and browse at the same time).

Your modem → dedicated line (not shared with neighbors) → DSLAM (at phone company's office) → ISP

- **Unlike cable, this line is dedicated to you** (not shared with neighbors), but the tradeoff is lower speed: 24–52 Mbps downstream, 3.5–16 Mbps upstream.

1.4 Home network

Everything at home (laptop, phone, smart TV) usually connects through one combo box that does several jobs at once: cable/DSL modem + router + firewall + NAT + Wi-Fi access point, feeding a wired Ethernet (up to 1 Gbps) and wireless Wi-Fi (54–450 Mbps) network inside your house.

1.5 Wireless access

Type	Coverage	Speed	Example
WLAN (Wi-Fi)	Small — inside/around a building (~100 ft)	11–450 Mbps	Your home router
Wide-area cellular	Large — 10s of km	10s of Mbps	4G/5G on your phone

1.6 Enterprise & Datacenter networks

- **Enterprise:** offices/universities use a mix of wired Ethernet (100 Mbps–10 Gbps) and Wi-Fi access points, connected through switches to an institutional router, which then connects out to the ISP.
- **Datacenter:** Companies like Google connect hundreds/thousands of servers together with very high bandwidth links (10s–100s of Gbps) so their servers can talk to each other and to the Internet.

1.7 Physical media — what the “wire” is actually made of

Guided media = signal travels through a solid material: - **Twisted pair (copper):** two insulated copper wires twisted together. Category 5 → 100 Mbps–1 Gbps; Category 6 → 10 Gbps. - **Coaxial cable:** two concentric copper conductors (one inside the other); can be split into multiple frequency channels (“broadband”), each carrying 100s of Mbps. - **Fiber optic:** glass fiber, data sent as pulses of light (a light pulse = 1 bit). Extremely fast (10s–100s of Gbps) and very low error rate because it’s immune to electrical/electromagnetic noise.

Unguided media = signal travels freely through the air (radio): - **Wi-Fi:** 10–100s of Mbps, works over 10s of meters. - **Wide-area cellular (4G/5G):** 10s of Mbps over ~10 km. - **Bluetooth:** short range, low speed — basically a cable replacement. - **Terrestrial microwave:** point-to-point (needs line of sight), ~45 Mbps. - **Satellite:** up to ~100 Mbps download, but has a huge ~270 ms delay one-way for geostationary satellites (because the satellite is very far away — signal has to travel up to space and back).

2. Switching — how data actually moves through the network core

Switching = deciding what path data takes through the network from source to destination. There are two fundamentally different approaches.

2.1 Circuit switching

A dedicated path is reserved end-to-end **before** any data is sent, and stays reserved for the whole conversation — even during silent gaps.

Analogy (book delivery): You want to send a whole book to a friend via a delivery service. With circuit switching, the courier reserves one vehicle and one route for a full hour just for you — even if you’re not sending anything at some point during that hour, nobody else can use that vehicle or route.

- Used in traditional (old) telephone networks.
- **Pro:** guaranteed, steady performance once set up (nothing else competes for that reserved capacity).
- **Con:** wastes capacity — if you pause, that reserved slot sits idle, unusable by anyone else.

2.2 Packet switching

The data is broken into small chunks called **packets**, and each packet is sent independently — possibly via different paths — then reassembled at the destination. **This is how the actual Internet works.**

Analogy (same book): Instead of sending the whole book at once, you tear out each page and send the pages separately (maybe via different couriers/routes). Your friend puts the pages back together in the right order once they've all arrived.

- **Pro:** No reserved-but-idle capacity — resources are shared efficiently (“statistical multiplexing”), no setup delay.
- **Con:** Congestion is possible (too many packets competing for the same link → delay/loss), and packets may arrive out of order, so they need to be reassembled and sometimes retransmitted.

2.3 Host sending function — how a message becomes packets

1. Host takes the full application message.
2. Breaks it into smaller chunks called **packets**, each of length **L bits**.
3. Transmits each packet into the network at the link's **transmission rate R** (also called link capacity/bandwidth).

Key formula:

$$\text{packet transmission delay} = \frac{L \text{ (bits)}}{R \text{ (bits/sec)}}$$

This is simply: *how long does it take to push the whole packet out onto the wire, bit by bit, at rate R?*

2.4 The network core: forwarding vs. routing

The **network core** is the mesh of interconnected routers in the middle of the network. It does two separate jobs, which are easy to confuse:

	Forwarding	Routing
Scope	<i>Local</i> action — happens inside one router	<i>Global</i> action — across the whole network
What it does	Looks at a packet's destination address in its header, checks its own local forwarding table, and moves the packet from the input link to the correct output link	Figures out (using routing algorithms) what the best source-to-destination <i>path</i> should be, and builds/updates the forwarding tables that routers use
Analogy	A signpost at one junction telling you which road to turn onto	The map-maker who decided the best overall route before any signposts were installed

2.5 Store-and-forward

Rule: A router must receive the **entire** packet before it can start forwarding any of it onward — it can't forward a packet bit-by-bit as it arrives; it has to “store” it fully first, then “forward” it.

One-hop worked example (from the slides): - L = 10 Kbits, R = 100 Mbps - One-hop transmission delay = $L/R = 10,000 \text{ bits} / 100,000,000 \text{ bps} = 0.1 \text{ ms}$

2.6 Queueing and packet loss

Every router output link has a **queue (buffer)** — packets waiting their turn to be transmitted. - **Queueing happens** when packets arrive at a link faster than the link can transmit them (arrival rate > R for some interval). - **Packet loss happens** when the buffer is completely full and a router simply can't hold any more waiting packets — the new arriving packet is dropped.

```
A ---\                (queue building up,
      \                waiting to go out on
B -----> [ router ] ==R=1.5Mb/s==> D
      /
C ---/
```

If A, B, and C are all sending fast, but the outgoing link can only push out 1.5 Mb/s, packets pile up in the queue — and get dropped if the buffer overflows.

3. Numerical Example — Circuit-Switched File Transfer

Question: How long does it take to send an 80 Kbyte file from host A to host B over a circuit-switched network, given: - All links run at 1.536 Mbps - Each link uses TDM (Time Division Multiplexing) with 24 slots/sec - Circuit setup time = 500 ms

Step-by-step solution:

Step 1 — Convert the file size to bits (networks always measure in bits, not bytes; 1 byte = 8 bits):

$$80 \text{ Kbytes} = 80,000 \text{ bytes} \times 8 = 640,000 \text{ bits}$$

Step 2 — Find the actual rate each circuit gets. The full 1.536 Mbps link is divided into 24 equal time slots (TDM), and your call only gets to use 1 of those 24 slots:

$$\text{Rate per circuit} = \frac{1,536,000 \text{ bps}}{24} = 64,000 \text{ bps}$$

Step 3 — Find the transmission time using our data amount and our actual circuit rate:

$$\text{Time} = \frac{640,000 \text{ bits}}{64,000 \text{ bps}} = 10 \text{ seconds}$$

Step 4 — Add the circuit setup time (remember: in circuit switching, you must reserve the path *before* sending anything):

$$10 \text{ sec} + 0.5 \text{ sec} = \mathbf{10.5 \text{ seconds total}}$$

4. Packet switching vs. circuit switching — capacity comparison

Setup: A 1 Gb/s link. Each user needs 100 Mb/s only when “active,” and is active only 10% of the time.

- **Under circuit switching:** Only **10 users** can be supported — because circuit switching must permanently reserve 100 Mb/s per user whether they're active or not, and $1 \text{ Gb/s} \div 100 \text{ Mb/s} = 10$.

- **Under packet switching: 35 users** can be supported, because with statistical sharing, the probability that more than 10 users are simultaneously active at once is under 0.0004 (a tiny risk) — so packet switching squeezes far more users onto the same link by betting that not everyone is “on” at the same time.

Takeaway: Packet switching wins big for “bursty” traffic (sometimes sending data, mostly idle) because it shares capacity instead of reserving it — at the cost of possible congestion (delay/loss) if too many users happen to burst at once, which is why protocols for reliable delivery and congestion control are needed on top.

Formula Summary — Lecture 02

Concept	Formula	Meaning
Packet transmission delay	$d_{trans} = \frac{L}{R}$	L = packet size (bits), R = link rate (bps)
Rate per circuit under TDM	$\frac{\text{Total link rate}}{\text{number of TDM slots}}$	Each user only gets 1 slot’s worth of the total capacity
File transfer time (circuit switching)	$\frac{\text{File size in bits}}{\text{Rate per circuit}} + \text{setup time}$	Don’t forget: bytes → bits (×8), and add setup time at the end

Lecture 03 — Chapter 1 (continued)

CS 3001 Computer Networks

1. Internet Structure: “A Network of Networks”

The question this section answers: With millions of individual access networks (ISPs) out there, how do they all end up connected to each other?

Let’s build this up step by step, the way the slides do:

Option 1 — connect every access ISP directly to every other access ISP. This doesn’t scale: with N access networks, you’d need on the order of N^2 connections. With millions of ISPs, that’s an impossible number of physical links.

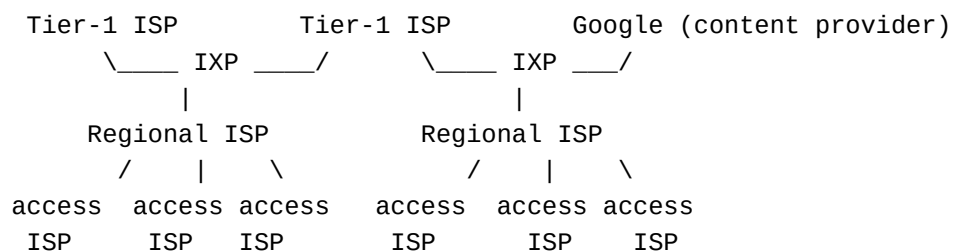
Option 2 — connect every access ISP to one single global transit ISP. Now everyone connects through one central hub, and customer ISPs pay that one global ISP (an economic/business agreement). Much better — but relying on a single company is risky (a monopoly, single point of failure).

Option 3 — multiple competing global ISPs (Tier-1 ISPs). If being “the global ISP” is a profitable business, competitors show up (e.g., Level 3, Sprint, AT&T, NTT). These giant ISPs then need to connect to *each other* too, at special meeting points called **IXPs (Internet Exchange Points)** — think of an IXP as a physical building where multiple big ISPs plug in their equipment to hand traffic to each other directly. When two networks agree to exchange traffic like this, it’s called a **peering link**.

Regional ISPs appear too — smaller ISPs that sit between local access networks and the big Tier-1 ISPs, aggregating traffic from a region before sending it upward.

Content provider networks (Google, Microsoft, Akamai, Facebook) build their *own* private networks connecting their datacenters directly to end users — often **bypassing** the Tier-1/regional ISPs entirely, to deliver content (like YouTube videos) faster and cheaper.

The resulting real-world structure



At the “center” sits a small number of very well-connected large networks (Tier-1 ISPs + big content providers), and everyone else (regional and access ISPs, i.e., you and me) connects up through them. This messy, economically-driven structure is why the real Internet’s map looks complicated — it evolved organically, not by a single master design.

2. Performance Metrics: Delay, Loss, Throughput

Now we ask: **how do we measure whether a network is “good”?** Three main metrics: **delay**, **loss**, and **throughput**.

2.1 Why delay and loss happen in the first place

Packets sit in router buffers (queues) waiting for their turn to be transmitted. If packets keep arriving faster than the router can push them out, the queue grows — and if the buffer (memory) fills up completely, new arriving packets get dropped (lost).

2.2 The four components of delay

Every packet, at every router (or “node”) it passes through, experiences up to four kinds of delay:

$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

Delay	What it is	Caused by	Typical size
d_{proc} processing delay	Time for the router to check the packet for bit errors and figure out which output link to send it on	Router internals	Usually tiny — microseconds
d_{queue} queueing delay	Time the packet sits waiting in the buffer for its turn on the output link	How congested the router currently is	Varies a lot — can be huge under congestion
d_{trans} transmission delay	Time to actually push the entire packet onto the link, bit by bit	Link properties	$d_{trans} = L/R$ (L = packet size in bits, R = link rate in bps)
d_{prop} propagation delay	Time for a bit, once pushed onto the wire, to physically travel across the link to the other end	Distance and the medium’s signal speed	$d_{prop} = d/s$ (d = link length, $s \approx 2 \times 10^8$ m/s for most media)

Important distinction — transmission delay vs. propagation delay (these are easy to confuse, so here’s the intuition): - **Transmission delay** = how long it takes to *squeeze* the whole packet out onto the wire (depends on packet size and link speed). - **Propagation delay** = once a bit is on the wire, how long it takes to physically *travel* the distance to the other end (depends only on distance and the speed of light/electricity in that medium — has nothing to do with packet size).

2.3 The Caravan Analogy (makes transmission vs. propagation delay click)

Imagine a 10-car caravan (= a 10-bit packet) traveling from one toll booth (= router) to another, 100 km away. - Each toll booth takes 12 seconds to let one car through (= bit transmission time). - Cars then travel at 100 km/hr on the open road (= propagation speed).

Q: How long until the whole caravan is lined up at the 2nd toll booth?

- **Transmission delay:** time to push all 10 cars through the first toll booth = $12 \text{ sec} \times 10 \text{ cars} = \mathbf{120 \text{ sec (2 minutes)}}$.
- **Propagation delay:** once the *last* car leaves the first booth, how long until it reaches the second booth 100 km away? $100 \text{ km} \div 100 \text{ km/hr} = \mathbf{1 \text{ hour (3600 sec)}}$.
- **Total time until the last car arrives at the 2nd booth** = 120 sec (waiting for all cars to be pushed through) + 3600 sec (that last car's travel time) = **3720 sec $\approx$ 62 minutes**.

Bottom line (from the extended example): the *first* bit arrives at the destination after (transmission delay + propagation delay) of the first hop. After that, **one more bit arrives every transmission-delay interval** (since they were pushed out back-to-back) — this is the idea behind **pipelining**.

Variant — what if propagation is faster than transmission? Now suppose cars travel at 1000 km/hr, and the toll booth takes 1 minute per car (much slower service). - Propagation delay = $100 \text{ km} \div 1000 \text{ km/hr} = 0.1 \text{ hour} = \mathbf{6 \text{ minutes}}$. - Since propagation (6 min) is now faster than the total service time for the caravan (would be 10 min), the **first car can arrive at the 2nd booth before the whole caravan even finishes being serviced at the 1st booth** — after 7 minutes, the first car has arrived at booth 2, while 3 cars are still waiting at booth 1.

Takeaway: whether propagation or transmission “dominates” the total delay depends entirely on the specific numbers (link speed vs. distance) — there's no universal rule; you must calculate both.

2.4 Worked Numerical — Segmentation and pipelining

Question: Host A sends a message $M = 10,000,000$ bits to host B, through one router R in between. Both links ($A \rightarrow R$ and $R \rightarrow B$) run at 10 Mbps, and propagation delay is 1 second on each link. Queueing and processing delay = 0. Find total delay for two scenarios:

Scenario 1 — sent as ONE big packet (no segmentation):

Step 1: Find transmission delay on each link (same formula both times, since both links have same L and R):

$$d_{trans} = \frac{L}{R} = \frac{10,000,000 \text{ bits}}{10,000,000 \text{ bps}} = 1 \text{ sec (per link)}$$

Step 2: Add up all four delay pieces across both hops (transmit onto link 1, propagate across link 1, then — because of store-and-forward — the router must wait for the **WHOLE** packet before transmitting onto link 2, then propagate across link 2):

$$\text{Total} = d_{trans1} + d_{prop1} + d_{trans2} + d_{prop2} = 1 + 1 + 1 + 1 = \mathbf{4 \text{ seconds}}$$

Scenario 2 — segmented into 10 equal packets of 1,000,000 bits each:

Step 1: Transmission delay for ONE packet on one link:

$$d_{trans} = \frac{1,000,000}{10,000,000} = 0.1 \text{ sec}$$

Step 2: Total delay for just the **first** packet to reach B (it still has to go through transmit-propagate-transmit-propagate, like before, just with a smaller packet):

$$0.1 + 1 + 0.1 + 1 = 2.2 \text{ sec}$$

Step 3: Now, because of **pipelining** — while the router is forwarding packet 1 onward, host A is already transmitting packet 2 behind it, and so on — the remaining 9 packets don't each need the full delay again. Each subsequent packet only adds one more transmission-delay's worth of "gap" behind the previous one:

$$9 \times d_{trans} = 9 \times 0.1 = 0.9 \text{ sec}$$

Step 4: Add it up:

$$2.2 + 0.9 = \mathbf{3.1} \text{ seconds}$$

Why is Scenario 2 faster (3.1s) than Scenario 1 (4s)? In Scenario 1, without segmentation, the router had to sit completely idle, waiting for the *entire* 10 Mbit message to fully arrive before forwarding even one bit onward (no overlap). In Scenario 2, the router can start forwarding the first small packet onward **while** host A is still transmitting the later packets — the work overlaps ("pipelines"), saving time.

2.5 Queueing delay and Traffic Intensity

Let **a** = average rate packets arrive (bits/sec into the queue), **R** = the link's service rate (bits/sec it can send out). Define:

$$\text{Traffic intensity} = \frac{La}{R}$$

(where L = average packet length, so La = arrival rate in bits/sec)

This ratio tells you how "loaded" the link is: - **$La/R \approx 0$** : arrivals are much slower than service — queue stays almost empty, average queueing delay is small. - **$La/R \rightarrow 1$** : arrivals are approaching the service rate — average delay grows large (queue keeps building up, barely draining). - **$La/R > 1$** : more work is arriving than the link can ever clear — the queue grows **without bound**, average delay is effectively infinite (this is congestion collapse).

Intuition: think of a checkout counter at a shop. If customers arrive slower than the cashier can serve them, the line stays short. As arrival rate approaches the cashier's max serving speed, the line gets longer and longer — and if customers arrive *faster* than the cashier can ever serve, the line grows forever.

2.6 "Real" Internet delay — traceroute

traceroute (tracert on Windows) measures the delay from your machine to *every* router along the path to a destination. It works by sending 3 probe packets meant to expire exactly at router 1, then 3 more for router 2, and so on — each router along the way sends a reply back, letting you measure the round-trip time to reach it.

- A * in the output means no response came back (the probe was lost, or that router chose not to reply).
- Delays don't always increase monotonically hop by hop — a later router can show a *smaller* delay than an earlier one, because delay is dominated by momentary queueing congestion at each router, not by total distance traveled.

2.7 Packet Loss

Every buffer (queue) before a link has a **finite capacity**. If a packet arrives when the buffer is completely full, it's dropped ("lost"). A lost packet might get retransmitted by the previous router, by the original sender, or not retransmitted at all — depending on what protocol is being used above it.

3. Throughput

Throughput answers a different question than delay: “At what rate is the destination actually receiving data from the source?”

3.1 Bandwidth vs. Throughput vs. Rate — don’t mix these up

People often use these words interchangeably in casual conversation, but they mean different things technically:

Term	Meaning	Analogy
Bandwidth	The <i>maximum theoretical</i> capacity of a link	Width of a highway — how many lanes exist (a theoretical ceiling, not what you’re actually getting)
Throughput	The <i>actual</i> amount of data successfully getting through, in a given time	How many cars actually make it through the highway per second, accounting for traffic jams, accidents, etc.
Rate	A general term for speed of sending/receiving — often used loosely, similar to bandwidth	—

3.2 End-to-end throughput and the bottleneck link

If data flows from a server through a link at rate R_s , then through a router, then through a second link at rate R_c to the client, the **overall throughput is limited by the slower of the two**:

$$\text{throughput} = \min(R_s, R_c)$$

The slower link is called the **bottleneck link** — the link on the end-to-end path that constrains (limits) the overall throughput, no matter how fast the other links are. (E.g., if $A \rightarrow R$ is 20 Mbps but $R \rightarrow B$ is only 10 Mbps, R has created a bottleneck — you can never get more than 10 Mbps end-to-end, however fast A can send.)

3.3 Network scenario — shared bottleneck

If 10 connections are all fairly sharing one backbone bottleneck link of rate R , each connection’s throughput is:

$$\min(R_c, R_s, R/10)$$

In practice, it’s often the sender’s or receiver’s own link (R_c or R_s) — not the shared backbone — that ends up being the real bottleneck for any one connection.

4. Bandwidth-Delay Product (BDP)

BDP = the amount of data that can be “in flight” (sent but not yet acknowledged) on the network at any one moment.

$$\text{BDP} = \text{Bandwidth (bits/sec)} \times \text{RTT (round-trip delay, in seconds)}$$

Motorway analogy: Bandwidth = how many lanes the motorway has (cars that can enter per second). Delay = how long one car takes to travel from start to end. BDP = total number of cars in transit on the motorway at any given instant = (cars entering per second) $\times$ (time each car spends traveling). Example: Bandwidth = 5 cars/sec, delay = 10 sec $\rightarrow$ BDP = $5 \times 10 = 50$ cars in transit at once.

Worked Example 1 — Satellite network: Given: Bandwidth = 512 kbit/s, RTT = 900 ms

Step 1 — convert both to consistent units (bits and seconds):

$$512 \times 10^3 \text{ bits/s}, \quad 900 \times 10^{-3} \text{ s}$$

Step 2 — multiply:

$$\text{BDP} = (512 \times 10^3) \times (900 \times 10^{-3}) = 460,800 \text{ bits} = 460.8 \text{ kbit} = \mathbf{57.6 \text{ kB}}$$

Worked Example 2 — Residential DSL: Given: Bandwidth = 2 Mbit/s, RTT = 50 ms

Step 1 — convert units:

$$2 \times 10^6 \text{ bits/s}, \quad 50 \times 10^{-3} \text{ s}$$

Step 2 — multiply:

$$\text{BDP} = (2 \times 10^6) \times (50 \times 10^{-3}) = 100,000 \text{ bits} = 100 \text{ kbit} = \mathbf{12.5 \text{ kB}}$$

Why BDP matters: it tells you how much data you should be allowed to send before waiting for an acknowledgment, in order to fully “fill the pipe” without leaving the link idle — this becomes very important later when studying TCP’s sliding window.

Formula Summary — Lecture 03

Concept	Formula	Notes
Total nodal delay	$d_{\text{nodal}} =$	Sum of all 4 delay types at one router
Transmission delay	$d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$ $d_{\text{trans}} = L/R$	L = packet size (bits), R = link rate (bps)
Propagation delay	$d_{\text{prop}} = d/s$	d = link length (m), $s \approx 2 \times 10^8 \text{ m/s}$
Traffic intensity	La/R	a = packet arrival rate, L = avg packet size, R = link rate. Must stay < 1 or delay $\rightarrow \infty$
End-to-end throughput (2 links)	$\min(R_s, R_c)$	Limited by the slower (bottleneck) link
Throughput with N shared connections	$\min(R_c, R_s, R/N)$	R = shared bottleneck link rate

Concept	Formula	Notes
Bandwidth-Delay Product	$BDP = \text{Bandwidth} \times \text{RTT}$	Amount of data “in flight” at once; remember to match units (bits & seconds) before multiplying

Lecture 04 — Chapter 1 (continued)

CS 3001 Computer Networks

1. Network Security

Why this matters: The Internet was never originally designed with security in mind — the original vision was “a group of mutually trusting users on a transparent network.” Because of that, security had to be bolted on *after* the fact, at every layer, and network engineers are still “playing catch-up” against attackers.

1.1 Malware — the “bad stuff” that rides along with the “good stuff”

When your device connects to the Internet, it can receive good content (search results, streaming) — but also malicious software (**malware**) that can: - Delete files, install spyware, and steal sensitive info (passwords, etc.) and send it to attackers. - **Self-replicate**, meaning it can spread from device to device on its own, exponentially fast.

Botnet: If your device gets infected, it can be silently enrolled into a **botnet** — a network of thousands of similarly-infected devices (“bots” or “zombies”), all secretly controlled by an attacker (the “bot-herder”) through a command-and-control server. This can happen without you ever noticing. The bot-herder can then use this army of devices to send spam, launch attacks, or even mine cryptocurrency using your device’s resources.

1.2 Denial of Service (DoS)

Definition: An attacker makes a resource (a server, or a link’s bandwidth) unavailable to legitimate users by flooding it with fake (“bogus”) traffic.

Typical steps: 1. Attacker selects a target. 2. Attacker breaks into many hosts around the network (building a botnet). 3. Those compromised hosts all send traffic at the target simultaneously — this “many machines attacking one target” version is called **DDoS (Distributed Denial of Service)**.

Three categories of DoS attacks:

Type	What it does
Vulnerability attack	Sends carefully-crafted messages, in just the right sequence, to crash the target service/device directly (exploiting a bug)
Bandwidth flooding	Sends an overwhelming volume of packets so the target’s <i>links</i> get completely clogged — legitimate traffic simply can’t get through
Connection flooding	Opens a huge number of half-open or fully-open TCP connections with the target, so it gets bogged down managing fake connections and can’t accept real ones

1.3 Packet Sniffing (eavesdropping)

On a **broadcast medium** (like shared Ethernet or Wi-Fi, where everyone’s traffic technically passes by everyone else’s device), a device running in “promiscuous mode” can read and record **all** passing packets — including ones not addressed to it, potentially capturing passwords sent in plaintext.

```
A -----> C           (A sends to C legitimately)
  \
   \--> B (B is "listening in" / sniffing the traffic on the shared medium)
```

This is exactly what **Wireshark** does (a free, legitimate packet-sniffing tool used in networking labs to inspect traffic for learning/debugging purposes).

1.4 IP Spoofing (fake identity)

An attacker injects a packet with a **false source address** — pretending the packet came from someone else (B), when really it came from the attacker.

```
Real sender: Attacker --[packet labeled "src: B, dest: A"]--> A
```

A receives a packet that *looks* like it came from B, but it actually came from the attacker — this can be used to impersonate trusted machines or to hide the attacker’s real identity.

1.5 Lines of defense

Now that we know the threats, here’s how networks defend against them — but each concept is a whole idea on its own, so let’s slow down and unpack each phrase individually:

- **Authentication** — *proving you really are who you claim to be*. Example: cellular networks use the physical SIM card inside your phone as hardware proof of your identity (nobody else has your exact SIM). The regular Internet doesn’t have an equivalent hardware-based proof built in by default — that’s part of why identity fraud online is easier than impersonating someone’s phone number.
- **Confidentiality (via encryption)** — *making sure that even if someone intercepts your data, they can’t read it*. Encryption means scrambling your data using a secret key before sending it, so that anyone sniffing the traffic (see 1.3 above) just sees gibberish — only someone with the correct key can unscramble (“decrypt”) it back into readable data. Confidentiality is the *goal* (keeping data secret); encryption is the *tool/technique* used to achieve that goal.
- **Integrity checks (via digital signatures)** — *making sure the data wasn’t tampered with in transit*. A digital signature is a small cryptographic “stamp” attached to a message; if even a single bit of the message changes after the signature was created, the signature no longer matches, so the receiver can detect (and reject) the tampering.
- **Access restrictions (via password-protected VPNs)** — *controlling who is even allowed to reach a resource in the first place*. A VPN (Virtual Private Network) creates a private, authenticated tunnel so only people with the right credentials can access certain internal network resources.
- **Firewalls** — specialized devices (“middleboxes”) sitting at the edge of access and core networks that filter traffic. They’re typically **“off-by-default”** — meaning they block everything except traffic they’ve been specifically told to allow — restricting which senders, receivers, and applications are permitted through, and can also detect and react to DoS attacks in progress.

2. Protocol Layers and Reference Models

The problem this solves: A real network has tons of moving pieces — hosts, routers, links, applications, protocols, hardware, and software, all interacting. How do we even talk about (design, discuss, troubleshoot) something this complex without getting overwhelmed? The answer is **layering**: breaking the whole system into a stack of separate layers, where each layer only has to worry about its own specific job.

2.1 Analogy — organizing air travel

Think about everything involved in flying with luggage: buying a ticket, checking baggage, boarding at the gate, taking off, the plane’s flight path, landing, unloading baggage, claiming baggage, and (if something goes wrong) filing a complaint. Instead of thinking about this as one giant tangled process, we can organize it into **layers**, where each layer is handled by its own dedicated **service**:

ticket (purchase)	→ ticketing service	→ ticket (complain)
baggage (check)	→ baggage service	→ baggage (claim)
gates (load)	→ gate service	→ gates (unload)
runway takeoff	→ runway service	→ runway landing
airplane routing	→ routing service	→ airplane routing

Each layer does its own internal work, and relies on the layer *below* it to provide its service — the gate service doesn’t care how the runway service works internally, it just trusts that “takeoff/landing” gets handled.

2.2 Why layering? (two big benefits)

1. **Explicit structure** — you can clearly identify each piece of the system and how the pieces relate to each other, giving everyone a shared reference model to discuss the system.
2. **Modularity (easy to maintain/update)** — if you change *how* one layer implements its service internally, the rest of the system doesn’t need to change at all, as long as that layer’s service to the layer above stays the same. (Example: changing exactly how gate boarding is organized internally doesn’t affect the ticketing or baggage systems at all.)

2.3 The Internet’s 5-layer protocol stack

Layer	Job	Example protocols
Application	Supports actual network applications — the layer your apps directly talk to	HTTP, IMAP, SMTP, DNS
Transport	Moves data between specific <i>processes</i> (running programs) on two hosts	TCP, UDP
Network	Routes data (as “datagrams”) from the source host all the way to the destination host, across possibly many routers	IP, routing protocols
Link	Moves data between two <i>directly-connected</i> neighboring devices (one hop)	Ethernet, 802.11 (Wi-Fi), PPP

Layer	Job	Example protocols
Physical	Moves the actual raw bits “on the wire” (or through the air)	—

Note: There’s just ONE network-layer protocol in practice (IP) — everything above and below it can vary (many app protocols, many link protocols), but IP is the one universal “glue” layer that lets all of them interoperate. This is sometimes called the “hourglass” shape of the Internet architecture.

2.4 Encapsulation — wrapping data as it goes down the stack

As data moves *down* through the layers on the sending side, each layer **wraps** (“encapsulates”) the data it receives from the layer above by adding its own **header** — a small chunk of extra control information specific to that layer’s job. Think of this exactly like **Russian nesting dolls (Matryoshka dolls)** — each doll wraps around the smaller one inside it.

```

Application:  [          M (message)          ]
Transport:    Ht [          M          ]      → called a "segment"
Network:      Hn Ht [          M          ]    → called a "datagram"
Link:         Hl Hn Ht [          M          ]  → called a "frame"

```

- **M** = the original message from the application (unchanged all the way through)
- **Ht** = transport-layer header (added by TCP/UDP) — used by the transport layer to implement its own service (e.g., making sure data arrives reliably)
- **Hn** = network-layer header (added by IP) — used to route the datagram from source host to destination host
- **Hl** = link-layer header (added by Ethernet/Wi-Fi) — used to move the frame across just the next single hop, to the next neighboring device

On the **receiving side**, this happens in reverse (**decapsulation**) — each layer strips off its own header and passes the remaining content up to the layer above it, until only the original message M is left for the receiving application.

2.5 Logical vs. Physical communication

This is a subtle but important distinction:

- **Logical communication:** Conceptually, we like to imagine each layer “talking directly” to the *same* layer on the other host — e.g., “the transport layer on Host A talks to the transport layer on Host B” — even though that’s not literally what happens physically.

```

Host A:  App ↔(logical)↔ App  Host B
Host A:  Transport ↔(logical)↔ Transport  Host B
... and so on for every layer

```

- **Physical communication:** In reality, data actually flows *down* through all the layers on Host A, across the physical wire, *up* through the relevant layers at each router it passes through (routers typically only need to look at layers up to Network — they don’t have an Application layer), and finally *up* through all the layers again at Host B.

```

Host A (App→Transport→Network→Link→Physical)

```

```

    ↓ down through the stack
Router (Network→Link→Physical only – routers just forward, don't run apps)
    ↓ down, then out
Host B (Physical→Link→Network→Transport→App)
    ↑ back up the stack

```

2.6 Chapter 1 Summary (what you should walk away with)

- Internet overview and what a protocol is
 - Network edge, access networks, and the network core (packet-switching vs circuit-switching, Internet structure)
 - Performance: loss, delay, throughput, BDP
 - Layering and how services build on each other via encapsulation
 - Security threats and defenses
 - (History, not covered in detail in these notes since it's non-technical background)
-

3. Additional Material

3.1 The OSI reference model (vs. the Internet's model)

The **OSI (Open Systems Interconnection)** model is an older, more theoretical 7-layer reference model, taught alongside the Internet's actual 5-layer model. It adds **two extra layers** that the real Internet stack doesn't use:

```

Application      7
Presentation     6 — OSI has these two extra layers;
Session          5 — the real Internet stack skips them
Transport        4
Network          3
Link (Data Link) 2
Physical         1

```

- **Presentation layer:** would let applications interpret the *meaning* of data — e.g., encryption, compression, or converting between machine-specific data formats (like different ways of encoding numbers).
- **Session layer:** would handle synchronization, checkpointing, and recovery of a data exchange (e.g., resuming a session after interruption).

Why the Internet doesn't have these as separate layers: the Internet's 5-layer stack simply doesn't include them — if an application actually needs this kind of functionality (encryption, compression, session management), it has to implement it itself *within* the application layer, rather than relying on a dedicated layer to provide it.

3.2 Wireshark — seeing the layers in action

Wireshark is a free packet-analyzer tool. It captures a copy of every Ethernet frame sent or received by your machine (via “packet capture,” saved in .pcap format), and lets you inspect it layer-by-layer — Physical → Link (Ethernet) → Network (IP) → Transport (TCP/UDP) → Application — so you can literally see the encapsulation/decapsulation process happening on real traffic. This is the tool used in the course's end-of-chapter labs.

Formula Summary — Lecture 04

No new numerical formulas were introduced in this lecture (it's conceptual: security + layering + encapsulation). The **Bandwidth-Delay Product** formula from Lecture 03 was briefly recapped at the start — see Lecture 03 notes for the worked examples.

Concept	Recap
BDP (recap from L3)	$\text{Bandwidth} \times \text{RTT}$ — data “in flight” at once
Encapsulation naming (memorize this!)	Application → message → Transport → segment → Network → datagram → Link → frame

Assignment #1 — Solutions

CS 3001 Computer Networks (5D & 5E) | Chapter 1

PART 1 — Textbook Questions (Kurose & Ross, 8th Edition, Chapter 1)

Review Question R12

Q: What advantage does a circuit-switched network have over a packet-switched network? What advantages does TDM have over FDM in a circuit-switched network?

Answer: A circuit-switched network can **guarantee a fixed amount of end-to-end bandwidth for the entire duration of a call** — once the circuit is set up, that capacity is reserved and yours alone. Most packet-switched networks today (including the Internet) **cannot make any such end-to-end bandwidth guarantee**, since capacity is shared dynamically and depends on how much other traffic happens to be competing for the same links at that moment.

TDM vs FDM: FDM needs **sophisticated analog hardware** to shift a signal into its assigned narrow frequency band and keep it isolated there. TDM avoids this — it just needs to know *when* (which time slot) to send, which is a much simpler digital operation, and each user gets to use the **full bandwidth** of the channel during their slot rather than being permanently boxed into one narrow frequency band.

Problem P3

Q: Consider an application that transmits data at a steady rate (the sender generates an N -bit unit of data every k time units, where k is small and fixed), and runs for a long time. **a)** Would a packet-switched or circuit-switched network be more appropriate for this application, and why? **b)** Suppose a packet-switched network is used, and the only traffic on it comes from applications like this one — and the sum of all the applications' data rates is less than the capacity of every link. Is congestion control needed? Why?

Answer: **a)** A **circuit-switched network** is better suited here. Because the application sends at a steady, predictable, smooth rate for a long session, bandwidth can be reserved for it up front without wasting capacity (there's no burstiness to accommodate). The one-time overhead of setting up and tearing down the circuit connection is also "amortized" (spread out) over the long duration of the session, so it's a small cost relative to the total transfer.

b) **No, congestion control is not needed** in this scenario. Even in the worst case — all applications transmitting at once — every link has *enough* capacity to handle the sum of all the applications' data rates simultaneously. Since demand never exceeds supply on any link, there's essentially no queuing, so there's nothing for congestion control to protect against.

Problem P6

Q: Consider two hosts, A and B, connected by a single link of rate R bps. The hosts are separated by m meters, and the propagation speed along the link is s meters/sec. Host A sends a packet of size L bits to Host B.

a) Express the propagation delay, d_{prop} , in terms of m and s . **b)** Determine the transmission time of the packet, d_{trans} , in terms of L and R . **c)** Ignoring processing and queuing delay, obtain an expression for the end-to-end delay. **d)** Suppose Host A begins transmitting at $t = 0$. At time $t = d_{trans}$, where is the last bit of the packet? **e)** Suppose d_{prop} is greater than d_{trans} . At $t = d_{trans}$, where is the first bit of the packet? **f)** Suppose d_{prop} is less than d_{trans} . At $t = d_{trans}$, where is the first bit of the packet? **g)** Suppose $s = 2.5 \times 10^8$ m/s, $L = 1500$ bytes, and $R = 10$ Mbps. Find the distance m so that d_{prop} equals d_{trans} .

Answer: **a)** $d_{prop} = m/s$ seconds — distance divided by propagation speed. **b)** $d_{trans} = L/R$ seconds — how long it takes to push the whole packet onto the link at rate R . **c)** $d_{end-to-end} = (m/s) + (L/R)$ seconds — the packet must first be fully pushed onto the wire, then travel the distance to Host B. **d)** At $t = d_{trans}$, the transmitting host has just finished pushing the packet onto the wire — so **the last bit is just leaving Host A**. **e)** If propagation takes longer than transmission, then at $t = d_{trans}$, that first bit — sent way back at $t = 0$ — hasn't had time to reach B yet: **the first bit is still on the link (in transit), not yet at Host B**. **f)** If propagation is faster than transmission, the first bit (sent at $t = 0$) has had *more* than enough time to cross the link by $t = d_{trans}$: **the first bit has already reached Host B**.

g) Step-by-step: We want $d_{prop} = d_{trans}$, i.e. $m/s = L/R$, so $m = \frac{L}{R} \times s$.

- $L = 1500$ bytes $= 1500 \times 8 = 12,000$ bits
- $R = 10$ Mbps $= 10 \times 10^6$ bps
- $s = 2.5 \times 10^8$ m/s

$$m = \frac{12,000}{10 \times 10^6} \times (2.5 \times 10^8) = \frac{1500 \times 8}{10 \times 10^6} \times (2.5 \times 10^8) = 3 \times 10^5 \text{ m} = \mathbf{300 \text{ km}}$$

Problem P10

Q: A packet of length L begins at end system A and travels over three links (connected by two packet switches) to a destination. Let d_i , s_i , R_i be the length, propagation speed, and transmission rate of link i ($i = 1, 2, 3$). Each packet switch adds a processing delay of d_{proc} . Assuming no queuing delays, find the total end-to-end delay symbolically, then compute it numerically for the given values.

Answer — symbolic formula:

At each link, the packet needs a **transmission delay** (to push it onto that link) plus a **propagation delay** (to physically cross that link); at each of the two intermediate switches, it also picks up a fixed **processing delay** before being forwarded onward:

$$d_{end-to-end} = \left(\frac{L}{R_1} + \frac{L}{R_2} + \frac{L}{R_3} \right) + \left(\frac{d_1}{s_1} + \frac{d_2}{s_2} + \frac{d_3}{s_3} \right) + 2 d_{proc}$$

(three transmission delays + three propagation delays + two lots of switch processing delay, since there are two switches along the path)

Numerical answer: Plugging in the given values (transmission delay per link = 4.8 ms each; propagation delays = 20 ms, 16 ms, 4 ms for the three links; processing delay = 3 ms at each of the 2 switches):

$$d_{end-to-end} = 4.8 + 4.8 + 4.8 + 20 + 16 + 4 + 3 + 3 = \mathbf{60.4 \text{ msec}}$$

Review Question R19

Q: Host A wants to send a large file to Host B. The path from A to B has three links in series, of rates $R_1 = 500$ kbps, $R_2 = 2$ Mbps, and $R_3 = 1$ Mbps. **a)** With no other traffic, what is the throughput for the file transfer? **b)** If the file is 4 million bytes, roughly how long will the transfer take? **c)** Repeat (a) and (b), but now with R_2 reduced to 100 kbps.

Answer:

a) Since the three links are all in *series* (one after another on the same path), the end-to-end throughput can never be faster than the **slowest (bottleneck) link**:

$$\text{Throughput} = \min(R_1, R_2, R_3) = \min(500, 2000, 1000) \text{ kbps} = \mathbf{500 \text{ kbps}}$$

b) Convert the file size to bits, then divide by the throughput:

$$\text{File size} = 4,000,000 \text{ bytes} \times 8 = 32,000,000 \text{ bits}$$

$$\text{Time} = \frac{32,000,000 \text{ bits}}{500,000 \text{ bps}} = \mathbf{64 \text{ seconds}}$$

c) With R_2 now reduced to 100 kbps, it becomes the new bottleneck:

$$\text{Throughput} = \min(500, 100, 1000) \text{ kbps} = \mathbf{100 \text{ kbps}}$$

$$\text{Time} = \frac{32,000,000 \text{ bits}}{100,000 \text{ bps}} = \mathbf{320 \text{ seconds}}$$

PART 2

Question 1 — Delay terms matching

Term	Definition
Processing delay	Time needed to perform an integrity check, look up the packet's information in a local table, and move the packet from an input link to an output link in a router.
Queueing delay	Time spent waiting in packet buffers for link transmission.
Transmission delay	Time spent transmitting packet bits into the link.
Propagation delay	Time needed for bits to physically propagate through the transmission medium from one end of a link to the other.

Question 2 — Security terms matching

Term	Definition
Firewall	Specialized “middleboxes” filtering or blocking traffic, inspecting packet contents.
Encryption	Provides confidentiality by encoding contents.
Digital Signatures	Used to detect tampering/changing of message contents, and to identify the originator of a message.
Access Control	Limiting use of resources or capabilities to given users.
Authentication	Proving you are who you say you are.

Question 3 — Store-and-forward switch timing

Setup: A —(2 Mbps, 25 us prop)— Switch —(2 Mbps, 25 us prop)— B. Switch adds 40 us processing delay per packet. 12,000 bits sent as 3 packets of 4,000 bits each.

Step-by-step: - Number of packets: $N = 12,000/4,000 = 3$ packets (P1, P2, P3) - Transmission delay per packet, per link: $d_{trans} = L/R = 4,000/2,000,000 = 2,000 \mu s$

Tracking packet 1 (P1):

Event	Time
A begins transmitting P1	0 us
A finishes transmitting P1	2,000 us
P1 fully arrives at Switch (+ propagation)	$2,000 + 25 = 2,025$ us
Switch finishes processing P1 (+ 40 us)	$2,025 + 40 = 2,065$ us
Switch finishes transmitting P1 onto Link 2 (+2,000 us)	$2,065 + 2,000 = 4,065$ us
P1 fully arrives at Host B (+ propagation)	$4,065 + 25 = \mathbf{4,090}$ us

Remaining packets (P2, P3): Since the transmission delay (2,000 us) is *larger* than the processing delay (40 us), each next packet is fully processed by the switch **before** the switch even finishes transmitting the previous one — so the link between switch and B never sits idle; packets flow through it back-to-back: - Switch finishes transmitting P2: $4,065 + 2,000 = 6,065 \mu s$ - Switch finishes transmitting P3: $6,065 + 2,000 = 8,065 \mu s$ - P3 (the last bit of all data) reaches Host B: $8,065 + 25 = \mathbf{8,090 \mu s}$

General formula used: Total Time = $(2 d_{trans} + 2 d_{prop} + d_{proc}) + (N - 1) d_{trans} = (4,000 + 50 + 40) + (2 \times 2,000) = 4,090 + 4,000$

Final Answer: 8,090 microseconds = 8.09 milliseconds

Question 4 — Queueing delay (R = 1,100,000 bps, L = 8,400 bits)

1. Does queueing delay vary a lot in practice? Yes. The formula $I(L/R)(1 - I)$ gives a rough theoretical *average*, but real traffic is bursty and unpredictable — actual instantaneous queueing delay swings a lot around that average, and it’s far more complicated in real networks than this simplified model suggests.

2. $a = 21$ packets/sec:

$$I = \frac{La}{R} = \frac{8400 \times 21}{1,100,000} = 0.1604$$

$$\text{Queueing delay} = I \cdot \frac{L}{R} \cdot (1 - I) = 0.1604 \times \frac{8400}{1,100,000} \times (1 - 0.1604) \times 1000 = \mathbf{1.0284 \text{ ms}}$$

3. $a = 58$ packets/sec:

$$I = \frac{8400 \times 58}{1,100,000} = 0.4429$$

$$\text{Queueing delay} = 0.4429 \times \frac{8400}{1,100,000} \times (1 - 0.4429) \times 1000 = \mathbf{1.8842 \text{ ms}}$$

4. **Buffer is infinite, queueing delay = 1.8842 ms, 871 packets arrive — how many are in the buffer 1 second later?** The idea: roughly $1000 \text{ ms} / 1.8842 \text{ ms} \approx$ how many packets can clear out of the buffer in that 1 second:

$$\text{Packets left in buffer} = a - \left\lfloor \frac{1000}{\text{delay}} \right\rfloor = 871 - \lfloor 1000 / 1.8842 \rfloor = 871 - 530 = \mathbf{341 \text{ packets}}$$

5. **Buffer max size = 645 packets — how many of the 871 are dropped?**

$$\text{Dropped} = 871 - 645 = \mathbf{226 \text{ packets}}$$

Question 5 — DoS / DDoS conceptual

I) **4,000 compromised devices flood the victim; none directly contacts the victim “administratively”;** student says “not a DoS since attacker never directly contacted victim.” The student is incorrect. This is still (in fact, precisely) a **DDoS attack**. What defines the attack is that the victim’s resources are being *deliberately overwhelmed* by traffic — and that traffic is coming from many (4,000) compromised, distributed hosts (a botnet). The bot-herder (the attacker) doesn’t need to personally contact the victim at all; the identity of the person controlling the attack is entirely separate from the hosts actually generating the traffic. The absence of a “legitimate-looking session” from the attacker is irrelevant to the classification.

II) **Server overwhelmed by traffic from a single attacker-controlled host (no botnet).** Student A says “not DoS, no botnet.” Student B says “definitely DDoS, traffic overwhelming.” The first student is correct (in the sense that it’s not a DDoS) — **this is a (plain, single-source) DoS attack**, because all the malicious traffic comes from just *one* attacker-controlled host. A **DDoS** attack specifically requires traffic to come from **multiple, distributed** sources (like a botnet) — the “D” for Distributed is the whole point. The sheer *volume* of traffic alone doesn’t make something a DDoS; the number of distinct attacking sources does.

Question 6 — Circuit switching & packet switching capacity (20 students, 150 KB each)

a) **Per-circuit transmission rate under TDM:**

$$\text{Rate per circuit} = \frac{\text{Link capacity}}{\text{TDM slots}} = \frac{2,048 \text{ kbps}}{32} = \mathbf{64 \text{ kbps}}$$

b) **Time to transmit one student’s file (including setup):**

$$\text{File size} = 150 \text{ KB} = 150 \times 8 \text{ Kbits} = 1,200 \text{ Kbits}$$

$$\text{Transmission time} = \frac{1,200 \text{ Kbits}}{64 \text{ Kbps}} = 18.75 \text{ sec}$$

$$\text{Total time} = 18.75 + 0.25 \text{ (circuit setup)} = \mathbf{19.0 \text{ seconds}}$$

c) Total time for all 20 students' files to finish: The link supports up to 32 simultaneous TDM circuits, and only 20 are needed — so all 20 students get their own dedicated circuit **at the same time**, running fully in parallel with no contention between them. Since none of them has to wait for another, they all finish together:

$$\text{Total time} = \mathbf{19.0 \text{ seconds}} \text{ (same as a single student's transfer time)}$$

d) Max number of users under packet switching (100 kbps needed when active, active 20% of the time):

There are two valid ways to read “guaranteed... only while active,” so both are given:

- **Strict peak guarantee** (worst case — enough capacity for every user to be active simultaneously and still get their full guaranteed rate):

$$\text{Max users} = \left\lfloor \frac{2,048 \text{ kbps}}{100 \text{ kbps}} \right\rfloor = \mathbf{20 \text{ users}}$$

- **Average / statistical-multiplexing capacity** (relying on the fact that not everyone is active at once, on average):

$$\text{Average demand per user} = 100 \text{ kbps} \times 0.20 = 20 \text{ kbps}$$

$$\text{Max users} = \left\lfloor \frac{2,048 \text{ kbps}}{20 \text{ kbps}} \right\rfloor = \mathbf{102 \text{ users}}$$